

Rapport

Conception

Une fois le sujet du projet lu, nous nous sommes réunis dans une première réunion afin de décider de la direction à prendre. Plusieurs idées ont été confrontées, et la première version de notre fichier README.md, réalisée par nos soins, résume cette réunion : une structure de fichiers et de dépendances cohérente qui restera inchangée jusqu'à la fin du projet, et la structure "Map" qui sera notre structure principale contenant la majorité des informations d'un niveau.

L'imagination de cette structure a constitué une première difficulté. Nous devons prendre en compte les futures possibilités d'affichage en art ASCII avec la librairie ncurses, ainsi que la lecture des fichiers de niveau, faisant partie des critères. Nous avons donc décidé qu'une matrice en 2 dimensions était la manière la plus naturelle de représenter notre jeu. Cette matrice contiendrait, sous forme d'entiers, des informations sur le niveau telles que la position des murs, des cibles, du vide (objet fixe) et potentiellement des caisses et des personnages, qui eux sont dynamiques.

Organisation

```
root
|
|--src
|  |-- main.c
|  |-- map.c
|  |-- lecture.c
|  |-- menu.c
|  |-- affichage.c
|
|--include
|  |-- utils.h
|  |-- map.h -> franck
|  |-- lecture.h *relativement court* -> yassine
|  |-- menu.h *plus tard*
|  |-- affichage.h *librairie ncurses* -> kian
```

Nous nous sommes alors fixé deux semaines pour conclure chacune de nos parties jusqu'à un certain niveau avec des fichiers de test pour chaque partie. Le fichier header `map.h` sera fait en premier afin de donner une base sur laquelle travailler aux deux autres, suivant très vite après le fichier source correspondant permettant des tests plus concrets. Les deux autres groupes ont alors pu pleinement travailler dans une branche Git dérivée de celle utilisée pour la structure de la carte.

Difficultés

Coté « Map »

La structure étant le cœur du projet, il est important qu'elle soit robuste et que chacun de ses aspects soit accessible aux autres utilisateurs. La multiplication du nombre de fonctions et de commentaires dans le fichier header de "map" est présente afin d'aider cet aspect.

Coté affichage

La découverte de la librairie ncurses, très peu manipulée jusqu'ici, a constitué l'une des premières difficultés. Bien que puissante pour faire fonctionner des interfaces utilisateur textuelles, elle présente des défis de compréhension et d'utilisation. La documentation, bien que riche, demande un certain temps d'adaptation avant de se familiariser avec les différentes fonctionnalités proposées par Ncurses. La gestion des fenêtres, des couleurs et des événements du clavier est particulièrement importante pour le bon fonctionnement du jeu.

Environnement

Dans deux cas, nous avons eu des problèmes liés à nos environnements de travail, comme l'impossibilité d'utiliser une distribution Linux ou même un sous-système Linux, pouvant demander un certain temps à se régler. La solution trouvée a finalement été d'utiliser les sessions de l'UFR à distance (par SSH) afin de dépanner. Elles ne permettent cependant pas d'utiliser la librairie ncurses. Ce problème nous a permis de faire nos recherches concernant les problèmes de compatibilités et d'environnement et les différentes méthodes d'y pallier.

Adresses

Une des difficultés du C qui s'est rencontrée très rapidement dans le projet a été la gestion des adresses, notamment la partie affichage dépendant de la carte.

La dernière difficulté a été de penser et mettre en œuvre les tests de robustesse dont le fonctionnement final était incertain jusqu'à très tard dans le projet.

Fonctionnalités

Le premier aspect à noter est la présence de la librairie ncurses avec tout un système de lecture et affichage des fichiers "texture" dédié. Le jeu se voit donc être plus propre et agréable à jouer avec une définition des fichiers textures modulables avec des modifications mineures dans le programme (et bien sûr le changement des textures de 3x5 en une autre définition).

Les tests de robustesse du fichier `main.c` permettent à n'importe qui d'écrire un fichier. Si celui-ci n'est pas valide, le niveau ne se lancera pas et le programme renverra une erreur explicitant le problème (les erreurs sont précisées dans le `README.md`). L'objectif étant que l'on ne rencontre jamais d'erreur en cours de jeu.

Conclusion

Au final, malgré les difficultés, le projet d'écrire un jeu Sokoban en C a porté ses fruits. Ces obstacles ont non seulement amélioré nos compétences en programmation, mais ont également permis de développer des stratégies efficaces pour résoudre des problèmes. Un processus d'apprentissage continu reste essentiel pour surmonter ces défis et progresser dans la réalisation des projets informatiques.